StretchBot: Neuro-Symbolic Planning for Adaptive Assistive Robot Guidance

Luca Vogelgesang[1], Mehdi Soltani[1], Xinrui Zu[1], Stefano De Giorgis[1], Mohammadhossein Khojasteh[1], Filip Ilievski[1*]

[1*]Faculty of Science, Department of Computer Science, Vrije Universiteit Amsterdam, De Boelelaan 1081a, Amsterdam, 1081 HV, Netherlands.

*Corresponding author(s). E-mail(s): f.ilievski@vu.nl;
Contributing authors: luca.vogelgesang@etu.umontpellier.fr; soltaniahmedmehdi@gmail.com; x.zu@vu.nl; stefano.de.giorgis@gmail.com; mohammadhosseinkhojaste@gmail.com;

Abstract

Assistive robots used in home and health settings need to adapt to user state and environmental context while preserving safe and coherent behavior. We present StretchBot, a hybrid coaching system for guided stretching routines. The implemented prototype combines a lightweight JSON knowledge graph, large language model (LLM) reasoning, object detection, and pose monitoring, with optional modules for speech and affect signals. Perception and session-state cues are assembled into a context package that is enriched with knowledge relations and used for adaptive action selection (e.g., `NEXT_EXERCISE`, `POINT_<OBJECT>`, `STOP_ROUTINE`). We report a pilot proof-of-concept evaluation with three participants comparing scripted and adaptive conditions. Descriptive results suggest higher perceived adaptability and object relevance in the adaptive condition, while scripted interaction remains competitive for smoothness and predictability. These pilot-level findings suggest the practical potential of actionable knowledge representation for grounding LLM reasoning into embodied robot behavior and motivate larger studies to evaluate generalizability and long-term outcomes.

Keywords: Neuro-Symbolic AI, Large Language Models, Knowledge Graphs, Assistive Robotics, Adaptive Planning, Human-Robot Interaction

1 Introduction

Robotic systems are increasingly deployed in domestic and healthcare environments to support human wellbeing through physical assistance, rehabilitation, and guided exercise routines [4, 19]. Among these, assistive stretching robots have shown potential for improving flexibility, reducing stress, and encouraging physical activity in daily life [15]. However, most existing systems remain scripted and inflexible, providing limited personalization to the user's physical state, emotional condition, or surrounding context [1]. This lack of adaptation can reduce engagement, hinder safety, and limit long-term adoption.

The fundamental challenge lies in balancing two competing demands: (1) *structured predictability*—a clear script that ensures comprehensive exercise coverage and user confidence, and

(2) *contextual responsiveness*—real-time adaptation to the user's current state, emotions, and environment. Purely scripted systems excel at predictability but fail to personalize. Fully reactive systems can adapt but risk incoherence and repeated exercises.

To address these challenges, this work investigates whether and how adaptive robotic guidance may provide perceived benefits over scripted routines in terms of personalization, naturalness, and contextual relevance. We examine to what extent multimodal perception combined with commonsense reasoning enables a robot to adapt its guidance to the user's state and environment. Building on this research question, we developed **StretchBot**, an adaptive robotic coach designed to guide users through a short morning stretching routine while dynamically adjusting its behavior to real-time multimodal cues.

StretchBot integrates perception, commonsense reasoning, and interaction control in a unified architecture, enabling context-aware adaptation of stretching plans. At its core lies a *hybrid reasoning module* that couples a domain knowledge graph with a large language model (LLM) for decision support [9, 17]. In the codebase, an additional verifier module is available as an optional safeguard stage to check response formatting and tone before execution [20].

To complement the technical system description, we report a small-scale pilot comparison between scripted and adaptive guidance. The goal of this preliminary study is not to provide statistically grounded validation, but to examine feasibility in real interactive sessions, collect early user impressions, and identify design trade-offs that motivate a larger controlled evaluation.

1.1 Research Questions and Contributions

Our work addresses three core research questions:

RQ1: Can a hybrid neuro-symbolic architecture (LLM + knowledge graph) enable effective plan adaptation in assistive robotics?

RQ2: How should actionable knowledge be represented and integrated with LLM reasoning to ground robot decisions in both perception and domain knowledge?

RQ3: Do users perceive adaptive robot guidance as preferable to scripted routines, and in which dimensions?

The main contributions of this work are:

1. An adaptive stretching robot framework integrating object detection, pose monitoring, dialogue input, and optional multimodal perception modules (emotion recognition, speech, affect signals), coupled with commonsense reasoning and interaction control for guided physical routines.
2. Integration of knowledge graph reasoning with large language model prompt design, enabling the robot to adapt exercise plans based on user state, environment, and prior interaction history.
3. A collaborative interaction protocol ensuring explicit user confirmation before all actions, balancing proactive adaptation with human oversight.
4. An exploratory pilot evaluation comparing scripted and adaptive robot guidance, combining subjective ratings and qualitative feedback to identify early usability trade-offs in assistive stretching interactions.
5. Analysis of strengths and limitations of hybrid LLM-KG planning for assistive robotics, with implications for future neuro-symbolic systems.

2 Related Work

Our work sits at the intersection of four research areas: assistive robotics, commonsense reasoning, multimodal interaction, and LLM-based planning.

2.1 Assistive Robotics for Physical Guidance

Robots have been explored as assistants for stretching and rehabilitation exercises, aiming to improve user wellbeing and engagement [1, 15]. Prior systems often rely on fixed scripts or predefined adaptation rules, which can limit responsiveness to user state and context. In contrast, StretchBot combines perception with knowledge-grounded LLM reasoning to adapt instructions during a session.

Key challenges in assistive robotics include ensuring safety while granting autonomy, personalizing guidance to individual needs, and maintaining user engagement over time. Our approach addresses these by coupling structured exercise baselines with adaptive branching points.

2.2 Commonsense Reasoning and Knowledge Graphs

Commonsense reasoning has been used to give robots better understanding of their environment and user needs [9]. Knowledge graphs provide structured, interpretable representations of commonsense knowledge, with public resources like ConceptNet [17] encoding broad everyday relations. Recent work [24] has shown that combining symbolic knowledge bases with language models can improve both reasoning transparency and action feasibility. In StretchBot's design, commonsense reasoning connects perceived user states—such as fatigue or thirst—with relevant object affordances and situation-specific suggestions, enabling the robot to decide proactively when to suggest a break or indicate a helpful resource (e.g., water, coffee).

Knowledge graphs provide interpretability and structured reasoning compared to unstructured language models alone. By coupling KGs with LLMs, we leverage both the explainability of symbolic systems and the flexibility of neural language models.

2.3 Multimodal Interaction in Human-Robot Interaction

Prior work has shown that combining multiple sensing modalities—spanning acoustic, visual, and physiological channels—improves the adaptability and perceived quality of human-robot interaction [7]. For example, emotion recognition from facial expressions and vocal cues provides complementary information about user state [16], while posture monitoring detects physical strain [6]. StretchBot includes optional multimodal affect channels (facial, vocal, transcript-based emotion recognition) alongside core perception components: posture monitoring via skeletal landmark tracking [11] and environmental object detection. In the reported pilot, reasoning and action selection were primarily driven by pose, detected objects, dialogue state, and user input, with emotional signals treated as optional context. When enabled, multimodal fusion is intended to increase robustness by combining multiple sensing channels.

2.4 LLM-Based Planning and Reasoning

Recent work has explored the use of large language models for robotic planning and decision-making [2, 5]. The seminal SayCan work [2] combined LLM-based reasoning with affordance detection to generate feasible action plans. Subsequent research [10] has demonstrated that retrieval-augmented generation—augmenting LLMs with structured knowledge bases—improves both factual grounding and reasoning transparency. These insights have revealed that while LLMs excel at flexible, context-aware reasoning, they require grounding mechanisms to keep suggestions aligned with domain constraints. StretchBot applies these principles to assistive robotics by coupling open-weight LLMs (DeepSeek-R1, Llama-3.3-Nemotron) with a domain-specific knowledge graph and an optional verification layer, grounding LLM outputs in structured knowledge and constrained action grammar to improve coherence and relevance.

Our work differs from prior LLM-for-robotics approaches primarily in emphasizing an *action-oriented knowledge representation* that is explicitly designed for inline runtime injection, rather than as a post-hoc context enrichment. This design allows concrete LLM outputs to be naturally mapped to embodied robot actions under lightweight but explicit constraints (action prefixes, relation filtering, object grounding), making the neuro-symbolic integration more practical and traceable for assistive tasks.

3 System Architecture

StretchBot is structured around a four-stage pipeline: Perception, Reasoning, Action Execution, and Planning & Adaptation. Each stage transforms information into increasingly higher-level representations, enabling the robot to adapt its behavior in real time. Figure 1 summarizes this closed control loop and the direction of information flow.

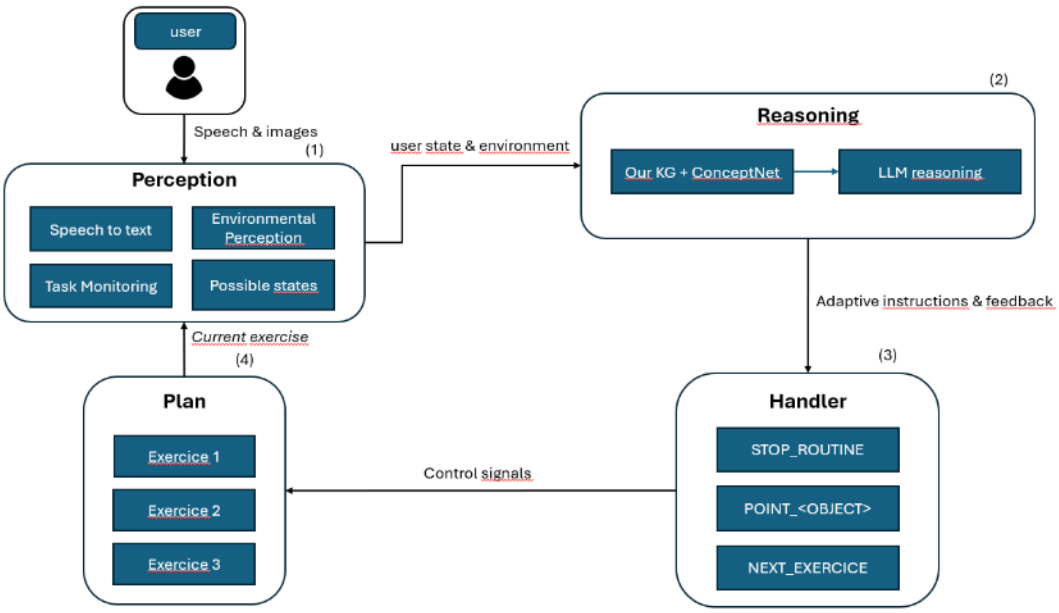

Fig. 1 Overall StretchBot pipeline organized into four stages: Perception, Reasoning, Action Execution (Handler), and Planning & Adaptation. The numbered markers indicate the main flow steps discussed in the text.

The numbered markers in Figure 1 correspond to the operational sequence used in our implementation. Marker **(1)** denotes the multimodal input transfer from the user to the *Perception* block (speech and image streams). Marker **(2)** denotes the transition from perceived user/environment state to the *Reasoning* block, where knowledge-graph and ConceptNet context are combined with large language model reasoning. Marker **(3)** denotes the delivery of adaptive instructions and feedback from *Reasoning* to the *Handler* (execution interface using actions such as `NEXT_EXERCISE`, `POINT_<OBJECT>`, and `STOP_ROUTINE`). Marker **(4)** denotes the control feedback from the Handler to the *Plan* block, which updates the current exercise stage and closes the adaptation loop.

3.1 Perception Module

In Figure 1, this module is the left block and corresponds to the first stage of the loop. The perception module provides the adaptive system with a continuous and multimodal understanding of both the user's state and the environment. It integrates four complementary components:

3.1.1 Object Detection

Visual scene analysis is performed using YOLOv8 [21], applied to the live RGB feed from the robot's camera. The system retrieves the complete list of all detected objects in the scene and sends this list to the reasoning module, ensuring that every recognized element is available for potential reasoning or adaptation.

3.1.2 Multimodal Emotion Recognition

The codebase includes three emotion-related channels:

- *Voice emotion recognition*—using a HuggingFace Transformers pipeline [23] on speech segments captured by the microphone.
- *Facial emotion recognition*—using DeepFace [16] applied to the video stream.
- *Text-based emotion recognition*—using a lightweight HuggingFace model [23] on transcribed speech.

In pilot runs, these channels were explored as optional context signals. The currently reported prototype behavior relies primarily on object/pose

context and dialogue state, with emotion fusion treated as a configurable extension. When enabled, emotion integration follows a late-fusion scheme: each channel provides an emotion prediction, reliability weights are applied across channels, and one aggregated emotion label is produced. This single fused label is then forwarded to the LLM context, rather than passing three potentially conflicting emotion outputs independently. Let $m \in \{v, f, t\}$ denote the voice, facial, and text channels, and let $p_m(e)$ be the confidence assigned by channel m to emotion class e. With non-negative reliability weights w_m, the fused score is computed as:

$$S(e) = \frac{\sum_{m \in \{v,f,t\}} w_m \, p_m(e)}{\sum_{m \in \{v,f,t\}} w_m}, \tag{1}$$

and the final emotion sent to the reasoning module is selected by

$$\hat{e} = \arg\max_{e} S(e). \tag{2}$$

3.1.3 Task Monitoring

Task monitoring verifies, in real time, that the user is correctly performing each stretching exercise and holding the required pose for the prescribed duration. It relies on **MediaPipe Pose** [11], a real-time skeletal estimation system that infers the positions of 33 body landmarks (shoulders, elbows, wrists, hips, ankles, etc.) from a single RGB frame at 30 Hz. This approach is preferred over earlier systems like OpenPose [6] for its efficiency and ease of integration in on-device pipelines.

Per-exercise pose criteria.

Each exercise is evaluated by a dedicated geometric check applied to the MediaPipe landmark coordinates:

- *Arms above the head.* The system checks that both wrists and both elbows have a y-coordinate strictly lower than the nose landmark ($y_{\text{wrist}} < y_{\text{nose}}$ and $y_{\text{elbow}} < y_{\text{nose}}$, where lower y means higher in the frame), and that the horizontal distance between the two wrists is below a threshold $w_{\max} = 0.3$ (in normalised image coordinates), ensuring the arms are raised and approximately centred above the head.
- *Touch-your-toes.* The system computes the Euclidean distance in normalised image coordinates between each wrist landmark and each ankle landmark:

$$d_{ij} = \sqrt{(x_{\text{wrist},i} - x_{\text{ankle},j})^2 + (y_{\text{wrist},i} - y_{\text{ankle},j})^2}\,, \tag{3}$$

  and accepts the pose when any $d_{ij} < 0.4$.
- *Lateral trunk lean (left / right).* The system computes the midpoint of the shoulders M_s and the midpoint of the hips M_h, then calculates the signed trunk inclination angle:

$$\alpha = \arctan\left(\frac{M_{s,x} - M_{h,x}}{M_{h,y} - M_{s,y}}\right), \tag{4}$$

  where the y-axis inversion accounts for OpenCV's top-down coordinate origin. The user is classified as leaning *right* when $\alpha > 15°$ and *left* when $\alpha < -15°$.

Hold-duration accumulation and reset.

At each valid frame the hold timer is incremented by the fixed frame period $\Delta t = 1/30\,\text{s}$, capped at the target duration $T = 5\,\text{s}$. If the user holds an invalid pose for longer than a reset tolerance of $t_{\text{reset}} = 40\,\text{s}$, the timer is zeroed and corrective spoken feedback is issued via the text-to-speech (TTS) channel. Once the timer reaches T, the task monitor emits a success event that propagates via markers **(3)** and **(4)** in Figure 1.

For the lateral lean exercise, two independent timers track the left-side and right-side hold durations; both must reach $T = 5\,\text{s}$ before the exercise is marked complete.

All pose checks are purely geometric and frame-local, requiring no additional training data beyond the pre-trained MediaPipe model.

3.1.4 Speech-to-Text (STT)

The codebase includes a Vosk-based STT component for local/offline speech transcription [22]. We selected Vosk for initial low-cost experimentation because it can be used locally without paid API calls. Google Cloud Speech-to-Text was considered as a strong alternative in early exploration [8], especially for potentially higher transcription robustness in real-world conditions; in a cloud setup, pairing stronger STT with neural TTS can also improve the overall perceived naturalness of

interaction. In the pilot runtime reported here, text input was used as a robust fallback during interactive testing.

Perception and interaction outputs are structured into a unified context package that can include detected objects, task execution status, dialogue input (typed or transcribed), and optional affect cues. This package operationalizes marker **(1)** and is transmitted to the reasoning block through marker **(2)** in Figure 1.

3.2 Reasoning Module

In Figure 1, this module is the upper-right block that receives the perception context through marker **(2)** and sends actionable output to the handler through marker **(3)**. The reasoning module serves as the central decision-making component, responsible for determining the robot's next action by combining perception outputs, prior knowledge, and the predefined stretching routine. Its operation follows a hybrid approach: rule-based checks handle explicitly defined conditions, while an LLM provides high-level reasoning, contextual adaptation, and natural-language interaction.

3.2.1 Context Integration and Knowledge Graph Retrieval

Each time the user speaks, the reasoning module compiles an updated context package including:

- the list of objects detected by perception,
- the aggregated emotional state (single fused label when multimodal affect is enabled),
- the current exercise progress status,
- the latest speech-to-text transcription.

To provide the LLM with meaningful background knowledge, this raw context is enriched with semantic relations retrieved from a knowledge graph (KG). A KG is a structured representation of entities and their relations, used here to encode contextual and actionable knowledge relevant to assistive robot behavior [9, 17]. Unlike unstructured text, a KG enables the system to reason over entities and their connections in a transparent and interpretable way.

3.2.2 Retrieval and Decision Policy (Implementation-Level)

To make the adaptive core explicit, we summarize the runtime policy used in the pilot code.

LLM-call trigger.

The reasoning cycle is activated when either (i) new user input is available or (ii) the current pose-monitoring status becomes `success`. If neither condition is true, the system continues low-level pose monitoring without issuing a new LLM call.

Concept set and KG query timing.

At each triggered cycle t, the candidate concept set is assembled as:

$$\mathcal{C}_t = \mathcal{C}_{\text{domain}} \cup \mathcal{C}_{\text{state}}(t), \tag{5}$$

where $\mathcal{C}_{\text{domain}}$ is a fixed domain list in the pilot script (e.g., `Coffee`, `Banana`, `GlassOfWater`, `Towel`, `Chair`) and $\mathcal{C}_{\text{state}}(t)$ contains user-state terms active at time t. For each concept $c \in \mathcal{C}_t$, the system first searches the local JSON KG; if no matching entity exists, it falls back to ConceptNet.

ConceptNet fallback policy.

Fallback retrieval is relation-filtered to a predefined subset (`UsedFor`, `Causes`, `HasProperty`, `MotivatedBy`, `IsA`) to reduce noise before prompt injection. The resulting relation bundle is serialized and passed to the LLM together with dialogue, exercise state, and detected objects.

Deterministic checks before and after LLM.

Before LLM invocation, deterministic modules provide: (a) exercise-completion status from pose monitoring, (b) hold-time progress, and (c) currently detected objects. After LLM generation, a rule-based parser extracts one actionable prefix (`NEXT_EXERCISE`, `POINT_<OBJECT>`, or `STOP_ROUTINE`) when present.

Action choice with multiple detected objects.

When multiple objects are visible, the LLM is prompted with the full list and returns a single `POINT_<OBJECT>` action if it chooses to point. The runtime then maps the object token to robot

command names (via a fixed mapping table for known aliases) and publishes one command. No additional deterministic re-ranking across multiple candidate objects is currently applied in the pilot script.

Hallucination and relevance safeguards.

The current safeguards are lightweight and layered: (1) constrained action grammar in the prompt, (2) explicit injection of observed object names, (3) action-prefix parsing before execution, and (4) an optional second-stage verifier module for format/tone correction. These mechanisms reduce malformed outputs but do not constitute a formal guarantee against all irrelevant suggestions, motivating stronger grounded decoding and verifier-by-default execution in future versions.

Concrete reasoning example.

To illustrate the integrated workflow, consider this interaction cycle: (1) User says *"I feel tired"* while the perception system detects a water bottle in the scene. (2) The system assembles the context package with `detected_objects = [GlassOfWater]`, `user_emotion = tired`, and `dialogue = "I feel tired"`. (3) A KG query retrieves relations: `Fatigue` $\xrightarrow{\text{DrinkWater}}$ `Water` and `Water` $\xrightarrow{\text{HasProperty}}$ `Refreshing`. (4) These relations are serialized into a structured prompt: *"User is tired. Objects available: water. Known relation: fatigue helps with water / water refreshes."*. (5) The LLM, prompted to adapt guidance to emotional state, generates: `POINT_WATER Want some water to refresh?`. (6) The action prefix parser extracts `POINT_WATER`, maps it to the robot's object-point action, and the arm points at the detected water bottle while playing the speech. (7) The cycle completes with an "action finished" event, allowing planning to check whether pose monitoring indicates success on the current exercise, or whether a new reasoning cycle is triggered by the next user input.

3.2.3 LLM-Based Adaptive Reasoning

The complete context package is passed to the primary LLM through a structured prompt. The model analyzes this information to decide the most appropriate next step in the stretching session, adapting the style and tone of its response to match the detected emotional state. If relevant objects are available in the scene, the model may refer to them directly in its suggestions.

The LLM adheres to predefined action prefixes in its outputs:

- `NEXT_EXERCISE:` instructs the system to move to the following stretch.
- `POINT_<OBJECT>:` directs the robot to point to a detected object.
- `STOP_ROUTINE:` ends the stretching session.

3.2.4 Response Verification

An additional verifier module is implemented in the repository to check response format, tone, and contextual coherence before execution. In the current prototype, this verifier is configurable and may be enabled or disabled depending on runtime constraints. When enabled, verifier-approved output is emitted as adaptive instructions and feedback (marker **(3)** in Figure 1).

3.3 Action Execution Module

This module corresponds to the *Handler* block in Figure 1. The action execution module grounds high-level reasoning decisions in concrete robot behavior. It acts as the interface between symbolic instructions and the physical world, ensuring that every action is carried out in a synchronized, safe, and comprehensible manner.

3.3.1 Command Interpretation

Each output from the reasoning stage arrives in dual format: a structured action prefix and a natural-language sentence. The action prefix encodes the intended behavior in machine-readable form, while the text provides a human-friendly explanation.

3.3.2 Physical Execution of Movements

Once a symbolic command such as `POINT_Coffee` has been issued, the system translates it into a fixed Cartesian target pose. The target is passed to MoveIt 2 [18], which computes a feasible joint-space trajectory while avoiding collisions. The trajectory is optimized using cubic spline interpolation [13] to ensure smooth and continuous motion.

The final optimized trajectory is executed step by step on the Lynxmotion SES900 arm [12]. Once the target pose is reached within tolerance, an "execution finished" event is published back to the reasoning module.

3.3.3 Verbal Feedback and Synchronization

In parallel with motion control, text-to-speech (TTS) converts the LLM's natural-language output into spoken instructions. The timing of speech and movement is carefully coordinated to enhance clarity and naturalness of the interaction [3]. The system supports multiple voices, ranging from simple rule-based synthesis to expressive neural voices capable of conveying emotion [14], reinforcing empathy and engagement.

3.3.4 Safety Supervision

User safety and hardware integrity are considerations in this prototype, addressed through conservative command design, bounded exercise routines, and operator oversight during pilot sessions. The broader architecture is designed to support additional runtime safeguards such as:

- Joint limits and velocity clamping constrain motion extrema.
- Collision detection via trajectory planning identifies potential obstacles.
- Emergency stop mechanism allows immediate halt if needed.

After execution status is updated, the handler sends control feedback to the planning block, implementing marker **(4)** in Figure 1.

3.4 Planning and Adaptation Module

In Figure 1, this module is the lower-left *Plan* block. It receives control feedback from the handler via marker **(4)** and maintains the current exercise state that is fed back into perception for the next cycle. The planning and adaptation module ensures that StretchBot delivers a stretching session that is both structured and responsive.

3.4.1 Global Routine Structure

At its core, the system relies on a predefined exercise script composed of a sequence of stretching primitives (e.g., arm stretch, side bend, neck rotation). Each primitive specifies the expected body posture, target hold duration (5 s), and the transition to the next exercise. This script acts as a baseline plan that guarantees coverage and maintains a clear flow.

3.4.2 Local Adaptations

On top of this baseline, the system introduces adaptive branching points where the flow can be adjusted depending on the user's state. For example:

- If the user reports fatigue, the system may suggest a brief pause or recommend a coffee break while maintaining the overall sequence.
- If the user expresses discomfort with a particular stretch, the system can substitute a gentler alternative or skip to the next exercise.
- If relevant objects are detected in the environment, the system proactively offers suggestions (e.g., "I see a bottle of water nearby—would you like a sip?").

These adaptations occur within the framework of the global routine, ensuring that the session remains coherent and goal-directed while accommodating individual preferences and states. Together, markers **(1)** to **(4)** therefore describe one complete adaptation cycle that readers can follow in parallel with this section.

4 Implementation

4.1 Software Stack

StretchBot is built on a modular software architecture using Robot Operating System 2 (ROS2) to coordinate communication between perception, reasoning, and execution components. Key libraries include:

- **Perception:** YOLOv8n ("You Only Look Once", version 8, nano variant) for real-time object detection; **MediaPipe Pose** for skeleton landmark extraction and pose verification; DeepFace for facial emotion recognition; HuggingFace Transformers pipelines for voice-segment and transcript-level emotion analysis.

- **Reasoning:** Custom Python modules for KG retrieval and context integration; OpenRouter-served open-weight models (DeepSeek-R1 family in the pilot script, with alternative model IDs configurable in `config.py`) for LLM-based planning; optional verifier integration via the HuggingFace Inference Client.
- **Execution:** ROS2 command publishing for symbolic actions; local **gTTS** speech synthesis; optional **Vosk** STT (offline) and optional expressive TTS through a HuggingFace Gradio space.
- **Spatial Grounding:** In the pilot, object positions were fixed and predefined. The repository architecture supports optional ArUco marker-based localization (4×4 50-dictionary, OpenCV) for future metric 3-D pose estimation when dynamic object tracking is required.
- **Hardware Communication:** ROS2 (`rclpy`) with a dedicated node (`llm_commander`) publishing symbolic commands on the `target_point` topic; low-level drivers for the Lynxmotion SES900 arm.

A key architectural decision is the **concurrent execution** of LLM reasoning and pose monitoring. The LLM interaction runs in a dedicated Python thread, while the main thread executes the real-time MediaPipe pose-tracking loop at 30 Hz. An `exercise_done` flag and a `stop_flag` dictionary coordinate termination between the two threads. In the current pilot script, interaction input is text-based; STT can be enabled as a runtime variant.

4.2 Prompt Engineering and Response Format

The primary LLM is instructed through a comprehensive system prompt that provides context about the user, the stretching routine, detected objects, and emotional state. The prompt explicitly instructs the model to use predefined action prefixes and to adapt its tone based on the user's emotional state. The format ensures consistent machine-readable output while maintaining natural language quality.

Example system prompt fragment:

```
You are a friendly and empathetic
stretching assistant.
The user is currently [emotional_state].
Detected objects in the room are:
[objects].
The current exercise is [exercise_name],
and the user status is [status].
Use one of these action prefixes when
relevant:
NEXT_EXERCISE, POINT_<OBJECT>, STOP_ROUTINE.
Always be encouraging and personalized.
Respond naturally, adapting your tone to
the user's emotion.
```

4.3 Knowledge Representation

The internal knowledge graph is implemented as a structured **JSON file** (`knowledge_graph.json`) parsed at runtime into Python dictionaries. Each entity entry stores a `relations` dictionary mapping predicate names to values or lists of values—a lightweight alternative to a full RDF triple store that keeps the system dependency-free while maintaining full expressiveness for the stretching domain.

Entity Organization. The KG organizes entities into five primary categories represented in JSON:

- *Activities*: Stretching, breathing, resting, hydration (with associated benefits and constraints).
- *Physical States*: fatigue-related and discomfort-related states (e.g., `Fatigue`, `Thirst`, `Tired`, `InPain`).
- *Environmental Objects*: `Coffee`, `GlassOfWater`, `Towel`, `Banana`, `Chair`.
- *Actions*: `DrinkWater`, `DrinkCoffee`, `EatBanana`, `TakeBreak`, `StopExercise`, `TakeASeat`.
- *Abstract Concepts*: BodyTemperature, BodyRelaxation, Performance, Motivation (with causal dependencies).

Relation Types. The KG encodes relation types used directly by retrieval and prompting, including:

1. *Causal relations*: `causes`, `reduces`, `increases` (e.g., `fatigue -> reduces -> performance`).
2. *Affordance and utility relations*: `affords`, `used_for`, `requires`, `helps_with`.
3. *State and taxonomy relations*: `is_a`, `has_property`, `motivates`, `is_relevant_when`.
4. *Session and routine relations*: `contains`, `targets`, `goal`, `requires_flexibility`.

Actionability Grounding. Actionability is represented implicitly through relation patterns (for example, state-to-action links such as `Thirst -> motivates -> DrinkWater` and object-to-action links such as `GlassOfWater -> affords -> DrinkWater`). Retrieved relations are serialized into prompt context, and the LLM maps this context to executable action prefixes.

Scalability and Fallback Strategy. When an entity is not found in the internal KG, the system can query ConceptNet (`/c/en/<entity>`) and filter returned edges to relation types relevant to assistive interaction (for example, `UsedFor`, `Causes`, `HasProperty`, `MotivatedBy`, and `IsA`). This hybrid design acknowledges that a curated KG does not cover all everyday contexts while preserving interpretability through explicit relation filtering.

4.4 Hardware Setup

The prototype was developed for indoor assistive interaction with a camera stream, speech output, and ROS2-based action dispatch to robotic control software. In the reported pilot, object positions were predefined and fixed; the repository includes optional scripts for ArUco-based camera calibration for future deployment scenarios with dynamic object localization.

4.4.1 Cost and Compute Footprint

The pilot prototype was intentionally developed under low-budget constraints. In practice, the robotic arm represented the main hardware cost, while most of the remaining software stack relied on free or open-source components (ROS2, OpenCV, MediaPipe, and open-weight or API-accessed language models). The runtime also operated on a modest, non-high-end PC, indicating that the architecture is feasible without specialized compute hardware for proof-of-concept deployment.

4.4.2 Calibration and Spatial Referencing

In the reported pilot, object positions were manually configured with fixed screen coordinates for each recognized item. This simplified approach was sufficient for the controlled setting and N=3 exploratory evaluation. For future deployment with dynamic scene changes, the repository includes OpenCV-based scripts for camera intrinsic calibration and optional ArUco marker-pose estimation to enable adaptive spatial referencing.

4.4.3 Experimental Environment

The pilot sessions were conducted in a controlled indoor setting with a participant-facing camera view and a small set of common objects (e.g., water bottle, coffee cup, banana, towel, chair) placed within view for contextual pointing suggestions.

5 Proof-of-Concept and Pilot Evaluation

5.1 Evaluation Design

We conducted an exploratory proof-of-concept user study contrasting a scripted baseline with the adaptive system. The study design was a within-subjects pilot comparison where participants experienced both conditions. This evaluation is intended as an initial feasibility and user-feedback study, not as a confirmatory validation of performance differences.

5.2 Participants and Protocol

Three participants (P1, P2, P3) aged 28–35 participated in the pilot study. Each participant completed two stretching sessions, each lasting approximately 8–10 minutes: (1) a scripted baseline condition where the robot followed a fixed exercise sequence without perception-based adaptation, and (2) the adaptive condition where the robot adjusted guidance based on user state and environment.

A two-minute break separated the two conditions. Participants were informed of the study's purpose and signed informed consent forms. The study was conducted in a controlled laboratory environment with standard objects present (coffee mug, water bottle, banana, chair).

5.3 Quantitative Metrics

After each condition, participants completed a custom questionnaire rating the robot's performance on six dimensions:

1. **Clarity:** How clear were the robot's instructions? (1–5 Likert scale)
2. **Comfort:** How comfortable were you during the routine? (1–5)
3. **Adaptability:** How well did the robot adapt to your state and needs? (1–5)
4. **Trust:** How much did you trust the robot's decisions? (1–5)
5. **Naturalness:** How natural and human-like was the robot's behavior? (1–5)
6. **Object Relevance:** How relevant were the robot's object suggestions? (1–5, N/A for scripted condition)

5.4 Qualitative Feedback

After completing both conditions, participants participated in semi-structured interviews (15–20 minutes) discussing:

- Which version they preferred and why.
- Perceived strengths and weaknesses of each approach.
- Suggestions for improvement.
- Overall sense of engagement and repeated-use intentions.

Qualitative feedback was summarized from participant interviews and used to identify recurring perceptions of strengths, limitations, and user preferences between scripted and adaptive interaction modes.

5.5 Planned Analysis for a Full-Scale Study

To move beyond pilot-level evidence, the next evaluation phase will use a pre-registered analysis plan. The primary endpoint will be perceived adaptability, with secondary endpoints including trust, naturalness, completion time, and intervention count. For subjective outcomes, we will use mixed-effects models with participant-level random intercepts to account for repeated measures. For non-normal outcomes (e.g., count-based interventions), we will use generalized mixed-effects models.

We will also report effect sizes and confidence intervals in addition to p-values, and include multiple-comparison correction for secondary outcomes. This design will allow us to test whether the trends observed in this pilot remain stable at larger scale while preserving ecological validity.

6 Results

6.1 Quantitative Results

Because the study involved only three participants in a controlled proof-of-concept setting, the results should be viewed as preliminary evidence rather than definitive confirmation. They provide useful early indications of user-perceived trends and trade-offs, while requiring validation in larger and more diverse evaluations.

The quantitative values reported below are computed from the participant Likert ratings summarized in Figure 2, where M denotes the mean score and SD denotes the standard deviation.

Clarity: Both conditions received favorable ratings for clarity (Scripted: $M = 4.67, SD = 0.33$; Adaptive: $M = 4.33, SD = 0.67$). The scripted version received a slight advantage, which may be attributable to its predictability. However, with only three participants, this trend should be interpreted cautiously.

Comfort: Comfort ratings were comparable between conditions (Scripted: $M = 4.33, SD = 0.67$; Adaptive: $M = 4.00, SD = 0.82$), suggesting that both systems were perceived as reasonably comfortable environments for stretching in this pilot.

Adaptability: In this small sample, the adaptive system received notably higher ratings for perceived adaptability ($M = 4.33, SD = 0.67$) compared to the scripted system ($M = 2.00, SD = 0.82$). This difference is suggestive and warrants further investigation with a larger and more diverse sample.

Trust: Trust ratings were comparable across conditions (Scripted: $M = 4.33, SD = 0.67$; Adaptive: $M = 4.00, SD = 0.82$), indicating that users reported similar confidence levels in both modalities.

Naturalness: The adaptive system received marginally higher ratings for perceived naturalness (Adaptive: $M = 4.33, SD = 0.33$; Scripted: $M = 3.67, SD = 0.89$), pointing to the possibility of more human-like interaction, though the small sample size limits generalizability.

Object Relevance: In the adaptive condition, object suggestions received high relevance

ratings ($M = 4.67, SD = 0.33$), indicating that participants found the KG-LLM grounded suggestions meaningful and appropriate.

Figure 2 visualizes the rating profile of both conditions and highlights the same descriptive trend observed in the numerical analysis: in this pilot sample, the adaptive condition received higher ratings for *adaptivity perception* and *relevance of object pointing*, while *clarity* and *comfort* remained similar or slightly in favor of the scripted mode. This pattern suggests that adaptation can improve perceived contextual relevance, but does not automatically guarantee a smoother interaction experience.

6.2 Qualitative Findings

Preference: Two participants (P2, P3) preferred the scripted version, citing predictability, smoother flow, and lower cognitive load. One participant (P1) preferred the adaptive version, appreciating its personalization and environmental awareness.

Strengths of Adaptive System: Participants noted that the adaptive system was "more thoughtful," "better at understanding my needs," and "made me feel heard." P1 specifically appreciated the robot's suggestion to point to the water bottle when noticing signs of dehydration.

Strengths of Scripted System: The scripted version was perceived as "more natural and smooth," with "no delays," and "easier to follow." Participants appreciated not having to answer questions or make decisions mid-routine.

Limitations of Adaptive System: P2 and P3 reported that "too many decisions made the session less relaxing," and that they "wanted to just follow along without thinking." There were occasional instances where LLM responses seemed slightly off-topic or required clarification.

Safety and Trust: All participants reported feeling safe in both conditions. We did not observe participant-reported unsafe suggestions in this pilot.

Across interviews, qualitative evidence clarified why ratings diverged between conditions. Participants describing the adaptive system as "more thoughtful" and "made me feel heard" associated these impressions with context-specific recommendations and object-aware prompts, suggesting that adaptation improved perceived relevance. By contrast, participants preferring the scripted mode emphasized flow continuity ("more natural and smooth") and reduced decision burden ("easier to follow"), indicating that adaptivity can also introduce interaction friction when users expect a low-cognitive-load routine. This trade-off between contextual intelligence and interaction smoothness is a central outcome of the pilot.

6.3 Technical Observations

The following observations emerged from the pilot implementation. However, given the small sample size ($N = 3$) and single experimental session per participant, these findings should be viewed as exploratory rather than confirmatory:

- **KG vs. ConceptNet Usage:** Internal inspection of pilot runs suggested that many responses were grounded in the internal KG, while a subset required ConceptNet fallback. This indicates that the curated KG covered common stretching-domain concepts, with broader commonsense retrieval helping for less common inputs.
- **LLM Verification Rate:** In exploratory runs where the verifier module was enabled, some primary outputs were reformulated (for example, tone or action-prefix normalization), suggesting potential value at additional latency cost.
- **Response Latency Under Low-Budget Setup:** In the pilot's low-budget configuration (modest local machine and API-based reasoning), response delays were sometimes noticeable and could occasionally reach roughly 3–10 seconds before coach feedback was produced. Participants generally tolerated this in a proof-of-concept context, but such pauses can reduce immersion during guided interaction.
- **Multimodal Fusion Robustness:** Optional multimodal affect cues were explored, but robust quantitative validation of fusion behavior was outside the scope of this pilot.
- **Safety Monitoring:** No adverse events or safety incidents were observed during the reported pilot sessions.

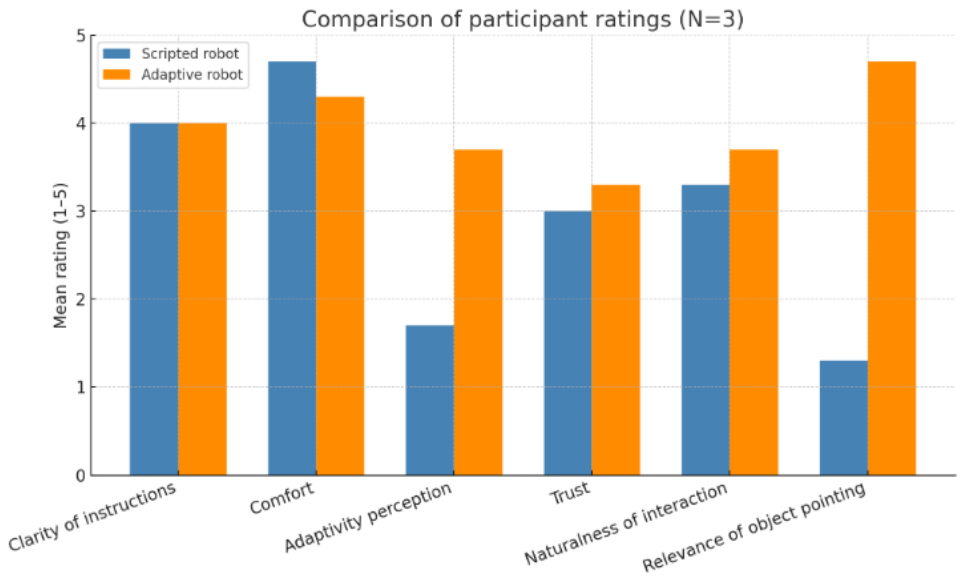

Fig. 2 Comparative results between scripted and adaptive conditions across the main user-evaluation dimensions.

7 Discussion

This discussion interprets the pilot findings as exploratory signals rather than full empirical validation. Their main value is to show that the prototype can be deployed in interactive sessions and that participants noticed meaningful differences between scripted and adaptive behavior.

7.1 Key Findings

Dimension-Specific Effectiveness: In the adaptive condition, the system showed promise for adaptability and environmental grounding, supporting the hypothesis that LLM + KG integration can enable context-aware decision-making in this pilot setting. However, this preliminary trend came at a potential cost in cognitive load for some users: two of three participants reported finding the scripted version more relaxing and natural. These findings, while interesting, require validation in larger-scale studies before broader claims can be made.

Actionable Knowledge Representation: In our pilot study, the internal KG appeared effective for common scenarios (fatigue, hydration, object affordances) but still required commonsense fallback for some novel situations. This supports the hypothesis that robust adaptation may require a hybrid representational backbone combining curated symbolic knowledge with broader commonsense resources, though larger studies are needed to assess generalizability.

LLM Verification Mechanisms: Exploratory runs suggest that a secondary verifier can improve format consistency and conversational appropriateness. However, the overhead (latency), runtime integration choices, and measured benefit require further systematic evaluation.

7.2 Interplay Between Structured and Adaptive Behavior

A key design choice in StretchBot is the balance between a global scripted baseline and local adaptive branching points. This hybrid approach appears to offer a middle ground: the global routine ensures comprehensive exercise coverage and predictability, while local adaptations provide personalization without complete unpredictability.

However, our results suggest that the optimal balance may be user-dependent. Users preferring predictability (P2, P3) might benefit from a "script-first" design where adaptation is offered optionally. In contrast, users seeking personalization (P1) might prefer adaptive-first with a safety net of fallback scripts.

7.3 Multimodal Perception and Emotion Recognition

Combining voice, facial, and text-based emotion signals is intended to reduce susceptibility to single-modality failures by producing one fused affect estimate for the reasoning stage. In this pilot, multimodal affect remained an optional context component and was not the dominant adaptation driver in all runs. Moreover, emotion recognition remains a coarse proxy for actual user state: a user might sound tired but actually be focused, and facial expressions can be ambiguous. Future work should benchmark fusion weights quantitatively and combine emotion with richer signals (e.g., biometric data, explicit user feedback).

7.4 Latency and Real-Time Constraints

In the low-budget pilot setup, adaptive responses were often perceived as slower than scripted execution, with occasional delays in the 3–10 second range before coach feedback was produced. For a proof-of-concept stretching routine, this remained manageable, but these pauses can disrupt interaction flow and perceived immersion.

A substantial share of this delay appeared to come from model/API communication and runtime overhead rather than low-level control loops. In future deployments, latency can likely be reduced through stronger hardware, lower-latency paid inference services, local model hosting, request batching, and lighter verification policies.

7.5 Limitations

1. **Small Sample Size:** With $N = 3$, our quantitative results cannot be generalized. A larger, multi-site study is needed to validate findings across diverse populations.
2. **Controlled Environment:** The study was conducted in a controlled laboratory with predefined objects. Real-world deployments may introduce variability that challenges the system's robustness.
3. **Short Interaction Duration:** The 8–10 minute stretching routines do not capture effects of long-term engagement or habituation. Longitudinal studies would be valuable.
4. **Limited KG Coverage:** The internal KG was hand-crafted for stretching scenarios. Scaling to diverse healthcare applications would require substantial KG expansion or automated KG construction.
5. **LLM Dependency:** The system's reasoning relies on LLMs accessed via external APIs (OpenRouter), introducing dependency on third-party services and variable latency. The current implementation uses open-weight models (DeepSeek-R1, Llama-3.3-Nemotron-49B) to reduce reliance on proprietary services. On-device models could further address this concern but may sacrifice reasoning quality.
6. **Evaluation Metrics:** We relied primarily on subjective Likert-scale ratings. Objective metrics (e.g., biometric measures of engagement, compliance, long-term health outcomes) would strengthen claims about system effectiveness.

7.6 Implications for Neuro-Symbolic Robotics

This work suggests that coupling LLMs with structured knowledge representations may offer a practical path toward more grounded and interpretable planning in assistive robotic settings. However, the integration is non-trivial: prompt engineering, knowledge graph design, and verification mechanisms must be carefully tuned. Moreover, the added complexity may not always be necessary; in some domains, scripted systems or purely neural approaches may suffice.

For assistive robotics specifically, the key insight is that *personalization and safety are not orthogonal*. By grounding LLM reasoning in domain knowledge and user perception, and by adding a verification layer, the proposed design aims to support adaptation while maintaining conservative safety constraints.

7.7 Practical Deployment Recommendations

Based on this pilot, we recommend four design principles for real-world deployment of adaptive assistive robots:

1. **Default to low cognitive load:** Start with a smooth scripted flow and introduce adaptive branching only when user state strongly indicates benefit.

2. **Expose adaptation controls:** Allow users to choose interaction style (scripted-first vs. adaptive-first) at session start and to change mode during execution.
3. **Separate safety from style:** Keep hard safety constraints deterministic (motion bounds, forbidden actions) while allowing conversational style to remain flexible.
4. **Log adaptation decisions:** Store structured traces of perception signals, KG evidence, and verifier edits to support auditability and post-hoc failure analysis.

These recommendations are intended to reduce friction in user experience while preserving the principal advantages of neuro-symbolic adaptation.

8 Conclusion and Future Work

This paper presented StretchBot, a neuro-symbolic adaptive robotic coach that integrates multimodal perception, knowledge graph reasoning, and large language models to guide personalized stretching routines. The primary contribution is the system design and implementation of a practical neuro-symbolic pipeline for grounded assistive guidance, while the pilot user study serves as a complementary exploratory assessment of feasibility and perceived interaction trade-offs.

Through a pilot user study with three participants, we explored the feasibility and user experience of adaptive versus scripted robot guidance in a stretching domain. Preliminary findings suggest that the adaptive system shows promise for improving perceived contextual relevance and environmental awareness, while scripted interaction remains competitive for perceived smoothness and predictability. These exploratory results motivate further research into user-centered design of adaptive assistive robots and suggest that the optimal interaction paradigm may be user-dependent.

8.1 Future Directions

1. **Extended User Studies:** Conduct larger, multi-site, longitudinal studies across diverse populations and health conditions to validate generalizability, robustness, and long-term effects of the adaptive approach.
2. **Richer State Representations:** Integrate biometric signals (heart rate, muscle tension, respiration rate) alongside emotions to provide more nuanced and physiologically grounded user state estimates.
3. **Scalability to Other Domains:** Extend the StretchBot framework to other assistive scenarios (e.g., rehabilitation after injury, cognitive exercises for elderly users, activity coaching for individuals with chronic conditions).
4. **On-Device Models:** Explore quantized LLMs, efficient transformer variants, and edge inference to reduce latency and reduce reliance on external API calls.
5. **Automated KG Construction:** Develop methods to automatically build, extend, and update domain-specific knowledge graphs from structured and unstructured data sources.
6. **User Preference Learning:** Implement reinforcement learning mechanisms to learn individual user preferences for scripted versus adaptive guidance over time, enabling dynamic mode switching.
7. **Formal Verification:** Apply formal methods and model-checking techniques to verify that the hybrid reasoning system maintains safety invariants and cannot issue unsafe or inappropriate commands.

8.2 Broader Impact

Adaptive assistive robots have significant potential to improve health outcomes and quality of life for aging populations and individuals with mobility or cognitive limitations. However, responsible deployment of such systems requires careful attention to user autonomy, informed consent, transparency in decision-making, and mechanisms for user control and override. This work represents a step toward principled, user-centered design of adaptive assistive robots, though real-world deployment requires ongoing engagement with ethics boards, healthcare practitioners, and end-users to ensure alignment with societal values and individual needs.

Acknowledgements. We thank all study participants for their time and feedback. We acknowledge support from the Vrije Universiteit Amsterdam and the Master's program in IRobotics at the University of Montpellier. Special thanks to

the discussion group members (Xinrui Zu, Stefano De Giorgis, Mohammadhossein Khojasteh) for valuable feedback throughout development.

Declarations

Funding

No external funding was received for this research beyond standard university support.

Competing Interests

The authors have no relevant financial or non-financial interests to disclose.

Ethics Approval and Consent to Participate

This pilot involved adult volunteers in a non-clinical stretching-assistance scenario. Participants were informed about study procedures and provided consent prior to participation.

Consent for Publication

Participants consented to the use of anonymized study results in research dissemination. Personal and directly identifying information was removed from reported data.

Data and Code Availability

Code and documentation for the prototype are intended to be made available in a public repository upon publication.

Anonymized pilot-level quantitative summaries and de-identified qualitative excerpts are available from the corresponding author upon reasonable request, subject to privacy constraints.

Raw audio/video recordings are not shared due to participant privacy.

Materials Availability

Not applicable.

Author Contribution

- **Luca Vogelgesang**: Project conception, system architecture design, implementation of reasoning module and knowledge graph, manuscript writing.
- **Mehdi Soltani**: Development of perception module (YOLO, emotion recognition, pose tracking), integration of multimodal sensor fusion, system testing and debugging.
- **Xinrui Zu**: Design and implementation of planning and adaptation module, trajectory optimization, ROS2 integration.
- **Stefano De Giorgis**: User study design, protocol development, data collection, qualitative analysis, critical manuscript review.
- **Mohammadhossein Khojasteh**: Knowledge graph construction, commonsense reasoning design, verification loop implementation.
- **Filip Ilievski**: Project supervision, research direction, knowledge representation framework, manuscript editing and final approval.

All authors contributed to the study conception, system development, and manuscript preparation. All authors read and approved the final manuscript.

References

[1] Aceituno-Valenzuela L, Núñez Marcos A, García-Zapirain Bn (2020) Robotic Coaching of Physical Exercise: A Systematic Review. Journal of Ambient Intelligence and Humanized Computing 11(8):3277–3290. https://doi.org/10.1007/s12652-019-01442-8

[2] Ahn M, Brohan A, Brown N, et al (2022) Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. arXiv preprint arXiv:220401691 Accepted to Robotics: Science and Systems (R:SS) 2022

[3] Breazeal C (2000) Toward Sociable Robots. IEEE Robotics & Automation Magazine 7(4):12–19. https://doi.org/10.1109/100.890603

[4] Broadbent E, Stafford R, MacDonald B (2012) Acceptance of Healthcare Robots for the Older Population: Review of Surveys. Health Informatics Journal 18(2):97–106. https://doi.org/10.1177/1460458211422192

[5] Brown TB, Mann B, Ryder N, et al (2020) Language Models Are Few-Shot Learners. Advances in Neural Information Processing Systems 33:1877–1901. NeurIPS 2020

[6] Cao Z, Hidalgo Martinez G, Simon T, et al (2019) OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields. In: 2019 IEEE/CVF International Conference on Computer Vision (ICCV), pp 7291–7299, https://doi.org/10.1109/ICCV.2019.00739

[7] D'Elia D, Stafford RQ, Broadbent E, et al (2014) Effects of Robot Expressiveness on Emotion Recognition. In: Proceedings of the 6th International Conference on Social Robotics (ICSR), pp 237–246, https://doi.org/10.1007/978-3-319-11973-1_24

[8] Google Cloud (2023) Google Cloud Speech-to-Text: Convert Audio to Text Using AI. URL https://cloud.google.com/speech-to-text, available at: https://cloud.google.com/speech-to-text

[9] Hogan A, Blomqvist E, Cochez M, et al (2021) Knowledge Graphs. ACM Computing Surveys 54(4):71:1–71:37. https://doi.org/10.1145/3447772

[10] Lewis P, Perez E, Piktus A, et al (2020) Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems, pp 9459–9474, neurIPS 2020

[11] Lugaresi C, Tang F, Nash H, et al (2019) MediaPipe: A Framework for Building Multimodal Machine Learning Pipelines. arXiv preprint arXiv:190608172 Accessed for pose estimation component in 2023

[12] Lynxmotion (2023) SES900 Dual Arm Robot Kit - Specifications and Documentation. URL https://www.robotshop.com/, available at: https://www.robotshop.com/

[13] Mellinger D, Kumar V (2011) Minimum Snap Trajectory Generation and Control for Quadrotors. In: 2011 IEEE International Conference on Robotics and Automation (ICRA), pp 2520–2525, https://doi.org/10.1109/ICRA.2011.5980409

[14] van den Oord A, Sifre K, Kalchbrenner N, et al (2016) WaveNet: A Generative Model for Raw Audio. arXiv preprint arXiv:160903499 Accepted to SSW 2016

[15] Robins B, Amirabdollahian F, Dautenhahn K, et al (2012) Therapeutic Robot for Managing Emotional Externalizing Behaviors in Children with Autism. In: Proceedings of the 4th International Conference on Social Robotics (ICSR), pp 143–152, https://doi.org/10.1007/978-3-642-34103-8_15

[16] Serengil SI, Ozpinar A (2016) DeepFace: Face Recognition via Deep Learning. In: 2016 IEEE 15th International Symposium on Innovations in Intelligent Systems and Applications (INISTA), pp 1–4, https://doi.org/10.1109/INISTA.2016.7571428, library also supports emotion/expression classification via facial feature extraction

[17] Speer R, Chin J, Havasi C (2017) ConceptNet 5.5: An Open Multilingual Graph of General Knowledge. In: Proceedings of the 31st AAAI Conference on Artificial Intelligence, pp 4444–4451

[18] Sucan IA, Chitta S (2013) MoveIt: An Open Source Robotics Manipulation Platform. IEEE Robotics & Automation Magazine 20(3):22–34. https://doi.org/10.1109/MRA.2013.2278551

[19] Tapus A, Mataric MJ (2007) Socially Assistive Robotics. IEEE Robotics & Automation Magazine 14(1):42–49. https://doi.org/10.1109/MRA.2007.339605

[20] Thawani A, Singh A, Prabhumoye S, et al (2021) Factual Error Correction for Abstractive Summarization Models via Reinforcement Learning. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 7720–7732, https://doi.org/10.18653/v1/2021.emnlp-main.604

[21] Ultralytics (2023) YOLOv8: A Real-Time Object Detection and Segmentation Model. In: GitHub Repository, URL https://github.com/ultralytics/ultralytics, available at: https://github.com/ultralytics/ultralytics

[22] Vosk Contributors (2021) Vosk: Offline Speech Recognition API. URL https://github.com/alphacep/vosk-api, open-source voice recognition kit. Available at: https://github.com/alphacep/vosk-api

[23] Wolf T, Debut L, Sanh V, et al (2019) HuggingFace's Transformers: State-of-the-Art Natural Language Processing. arXiv preprint arXiv:191003771 https://doi.org/10.18653/v1/2020.emnlp-demos.6, accepted to ACL 2020 (demo track)

[24] Zhang S, Roller S, Goswami A, et al (2022) Can Retrieval-Augmented Language Models Explain Their Predictions? In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pp 10884–10901, https://doi.org/10.18653/v1/2022.emnlp-main.746